INFORME TÉCNICO
CONTROL DE BRAZO ROBÓTICO DE 6 EJES CON ARDUINO UNO Y PCA9685

Santiago Perdomo
Santiago Corrales
Cristian Pinillo
Juan Diego Mondragon

Arquitectura de Computadores
Fernando Vélez Varela

Universidad Libre Seccional Cali
Facultad de Ingeniería
Cali, noviembre de 2026

CONTENIDO

INTRODUCCIÓN 3
1. OBJETIVOS 3
1.1 Objetivo general 3
1.2 Objetivos específicos 3
2. DESCRIPCIÓN DEL SISTEMA 3
2.1 Componentes de hardware 4
2.2 Flujo general de operación 4
3. ARQUITECTURA DEL CÓDIGO 4
3.1 Librerías y mapeo físico 4
3.2 Conversión de ángulo a pulso 5
3.3 Modos de movimiento y control de velocidad 6
4. PROTOCOLO DE COMUNICACIÓN SERIAL 8
5. LIMITACIONES Y MEJORAS 9
CONCLUSIONES 9
REFERENCIAS 10

INTRODUCCIÓN

Este informe documenta el sistema de control de un brazo robótico de seis grados de libertad que estamos armando a partir de un kit comercial. El control se hace desde un computador, que envía órdenes de texto por el puerto serial a un Arduino Uno. El Arduino no genera directamente las señales de los servomotores: delega esa tarea en un controlador PCA9685, al que se comunica por el bus I2C.

A lo largo del documento se describen los componentes, se explica el código por secciones, se detalla el protocolo de comandos y se señalan las limitaciones que identificamos al revisarlo.

1. OBJETIVOS

1.1 Objetivo general

Documentar y explicar el funcionamiento del firmware que controla un brazo robótico de 6 ejes mediante un Arduino Uno y un controlador de servos PCA9685.

1.2 Objetivos específicos

- Identificar los componentes de hardware y la asignación de cada servo a un canal del PCA9685.
- Explicar la conversión de ángulos a señales PWM de 12 bits.
- Describir los modos de movimiento y el control de velocidad.
- Especificar el protocolo serial entre el computador y el Arduino.
- Señalar las limitaciones del sistema y posibles mejoras.

2. DESCRIPCIÓN DEL SISTEMA

El sistema permite controlar de forma centralizada un brazo robótico de seis grados de libertad (DOF). El computador envía comandos de texto por comunicación serial a 9600 baudios; el Arduino Uno los interpreta y calcula la posición de cada articulación; el PCA9685, conectado por I2C, genera las señales PWM que reciben los servomotores.

Al generar el PWM por hardware, el Arduino solo necesita enviar el valor de cada canal cuando cambia, y el chip se encarga de mantener la señal. Esto simplifica el código y mejora la estabilidad de las señales frente a generarlas por software.

2.1 Componentes de hardware

Tabla 1. Componentes del sistema

Componente	Descripción	Función
Arduino Uno	Microcontrolador con interfaz USB	Recibe comandos seriales y ejecuta la lógica de movimiento
Shield PCA9685	Controlador PWM de 16 canales y 12 bits, con bus I2C	Genera las señales PWM de los servos
Servomotores	Tres de mayor tamaño y tres de menor tamaño, según el kit	Accionan las seis articulaciones del brazo
Fuente de alimentación externa	Adaptador de corriente incluido en el kit	Alimenta los servos sin cargar el puerto USB
Cable USB tipo B	Conexión Arduino–computador	Comunicación serial y programación
Estructura y tornillería	Piezas, tornillos y herramientas del kit	Ensamble mecánico del brazo

Nota. Elaboración propia a partir de los componentes del kit.

2.2 Flujo general de operación

1. El computador envía una línea de texto terminada en salto de línea por el puerto serial.
2. El Arduino la acumula en un buffer y, al detectar el salto de línea, la interpreta.
3. Según el comando, cambia de modo, ajusta la velocidad o actualiza la posición objetivo de un servo.
4. El Arduino convierte el ángulo a un valor de pulso y lo envía al PCA9685 por I2C.
5. El PCA9685 mantiene la señal PWM en el canal correspondiente y el servo se mueve.

3. ARQUITECTURA DEL CÓDIGO

El código se organiza en tres bloques: la configuración de librerías y canales, la conversión de ángulos a pulsos, y el control de velocidad y movimiento.

3.1 Librerías y mapeo físico

- `#include <Wire.h>`: habilita la comunicación I2C entre el Arduino y el PCA9685.

- `#include <Adafruit_PWMServoDriver.h>`: librería que facilita el manejo de los canales PWM de la tarjeta expansora.
- `Wire.setClock(400000)`: lleva el bus I2C a 400 kHz (modo rápido), lo que reduce el tiempo de actualización de los seis canales.

El arreglo `servoChannels[TOTAL_SERVOS] = {0, 1, 2, 3, 4, 5}` define el canal de la placa al que está conectado cada servo:

Tabla 2. Asignación de servos a canales del PCA9685

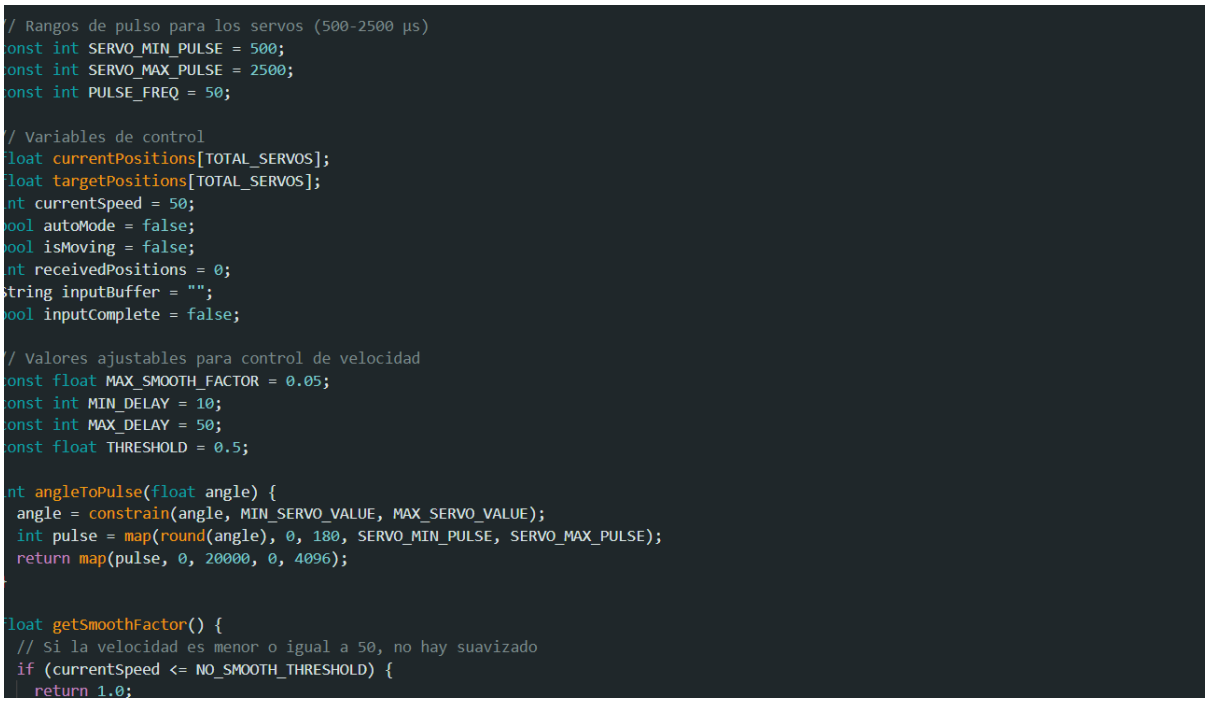
Servo	Canal	Articulación	Referencia en el código
1	0	Base	Comentario del código
2	1	Hombro	Según el montaje
3	2	Codo	Según el montaje
4	3	Muñeca	Según el montaje
5	4	Rotación de muñeca	Según el montaje
6	5	Garra (pinza)	Comentario del código

Nota. El código solo documenta de forma explícita el servo 1 (base) y el servo 6 (garra); las demás articulaciones corresponden al ensamble físico.

3.2 Conversión de ángulo a pulso

Los servomotores no interpretan grados sino anchos de pulso. La función `angleToPulse` recibe un ángulo, lo limita al rango de 0° a 180° y lo convierte en dos pasos. Primero lo traslada a un ancho de pulso entre 500 µs y 2500 µs. Luego lo convierte a ticks del PCA9685, cuya resolución es de 12 bits (4096 valores por periodo). Con una frecuencia de 50 Hz el periodo es de 20 000 µs, por lo que cada tick equivale a unos 4,9 µs.

Figura 1. Constantes, variables de control y función angleToPulse



```
// Rangos de pulso para los servos (500-2500 µs)
const int SERVO_MIN_PULSE = 500;
const int SERVO_MAX_PULSE = 2500;
const int PULSE_FREQ = 50;

// Variables de control
float currentPositions[TOTAL_SERVOS];
float targetPositions[TOTAL_SERVOS];
int currentSpeed = 50;
bool autoMode = false;
bool isMoving = false;
int receivedPositions = 0;
String inputBuffer = "";
bool inputComplete = false;

// Valores ajustables para control de velocidad
const float MAX_SMOOTH_FACTOR = 0.05;
const int MIN_DELAY = 10;
const int MAX_DELAY = 50;
const float THRESHOLD = 0.5;

int angleToPulse(float angle) {
 angle = constrain(angle, MIN_SERVO_VALUE, MAX_SERVO_VALUE);
 int pulse = map(round(angle), 0, 180, SERVO_MIN_PULSE, SERVO_MAX_PULSE);
 return map(pulse, 0, 20000, 0, 4096);
}

float getSmoothFactor() {
 // Si la velocidad es menor o igual a 50, no hay suavizado
 if (currentSpeed <= NO_SMOOTH_THRESHOLD) {
   return 1.0;
```

Nota. Captura del código fuente del proyecto.

Tabla 3. Valores de referencia de la conversión

Ángulo	Ancho de pulso	Valor enviado al PCA9685 (ticks)
0°	500 µs	102
90°	1500 µs	307
180°	2500 µs	512

Nota. Valores calculados a partir de las constantes del código (50 Hz, 4096 ticks por periodo).

Dentro de ese rango hay unos 410 ticks para 180°, es decir, una resolución cercana a 0,44° por tick, suficiente para este tipo de brazo.

3.3 Modos de movimiento y control de velocidad

El comportamiento depende de dos variables: el modo (manual o automático) y la velocidad (0 a 100).

Modo manual (`MODE 0`). Cada comando de posición se aplica de inmediato: el código actualiza la posición actual y escribe el PWM directamente. Es el único caso en que el servo salta a la posición pedida sin pasos intermedios.

Modo automático (`MODE 1`). El Arduino espera a recibir las posiciones de los seis servos. Cuando completa el sexto comando activa el movimiento y desplaza todos los servos en simultáneo hacia sus objetivos. Al terminar, envía `DONE` por el puerto serial.

Dentro del modo automático, la velocidad define cómo se desplaza cada servo:

- **Velocidad ≤ 50: movimiento lineal.** Cada servo avanza como máximo 1° por ciclo, sin suavizado. La velocidad se controla con la pausa entre ciclos (`getMovementDelay`), que va de 50 ms a 30 ms.
- **Velocidad > 50: movimiento suavizado.** El paso es proporcional a la distancia que falta (`diff * smoothFactor`) y está limitado por `maxStep = 2.0 * (velocidad / 100)`. El servo arranca con pasos mayores y reduce el paso al acercarse al objetivo, lo que evita frenadas bruscas sobre los engranajes.

Figura 2. Bucle principal: lectura de comandos y movimiento hacia el objetivo



```
void loop() {
  if (inputComplete) {
    processSerialCommand(inputBuffer);
    inputBuffer = "";
    inputComplete = false;
  }

  if (autoMode && isMoving) {
    bool allReached = true;
    float smoothFactor = getSmoothFactor();

    for (int i = 0; i < TOTAL_SERVOS; i++) {
      float diff = targetPositions[i] - currentPositions[i];

      if (abs(diff) > THRESHOLD) {
        allReached = false;

        if (currentSpeed <= NO_SMOOTH_THRESHOLD) {
          // Movimiento seco para velocidades <= 50
          float step = (diff > 0) ? min(diff, 1.0f) : max(diff, -1.0f);
          currentPositions[i] += step;
        } else {
          // Movimiento suavizado para velocidades > 50
          float step = diff * smoothFactor;
          float maxStep = 2.0 * (currentSpeed / 100.0);

          if (abs(step) > maxStep) {
```

Nota. Captura del código fuente del proyecto.

Un detalle del diseño es que `getSmoothFactor()` disminuye cuando la velocidad aumenta. La mayor rapidez no viene de ese factor, sino de que el tope `maxStep` crece y la pausa entre ciclos se acorta.

Tabla 4. Parámetros de movimiento según la velocidad

Velocidad	Tipo de movimiento	Pausa por ciclo	Paso máximo	Factor de suavizado
0	Lineal	50 ms	1°	No aplica

Velocidad	Tipo de movimiento	Pausa por ciclo	Paso máximo	Factor de suavizado
50	Lineal	30 ms	1°	No aplica
75	Suavizado	20 ms	1,5°	0,075
100	Suavizado	10 ms	2,0°	0,050

Nota. Valores calculados a partir de las fórmulas del código. La velocidad 0 no detiene el movimiento: corresponde a la velocidad lineal mínima.

El movimiento termina cuando todos los servos quedan a menos de 0,5° de su objetivo (`THRESHOLD`). En ese momento se desactiva `isMoving` y se envía `DONE`.

4. PROTOCOLO DE COMUNICACIÓN SERIAL

El brazo recibe líneas de texto a 9600 baudios. Cada comando termina en un salto de línea (`\n`); los caracteres se acumulan en un buffer en la función `serialEvent()` y se interpretan en `processSerialCommand()`.

Tabla 5. Comandos del protocolo

Comando	Formato	Efecto	Respuesta
Modo	`MODE 0` o `MODE 1`	0: manual. 1: automático (reinicia el conteo de posiciones recibidas).	Ninguna
Velocidad	`SPD n` (n de 0 a 100)	Ajusta la velocidad de desplazamiento. Valores fuera de rango se ignoran.	Ninguna
Posición	`s ángulo` (s de 1 a 6, ángulo de 0 a 180)	Define la posición objetivo del servo s. En manual se aplica al instante.	Ninguna
Fin de movimiento	Enviado por el Arduino	Indica que todos los servos alcanzaron su objetivo (solo en modo automático).	`DONE`

Nota. Elaboración propia a partir de `processSerialCommand()`.

Ejemplo de una secuencia coordinada en modo automático:

```
MODE 1
SPD 70
1 90
2 45
3 120
4 90
```

```
5 60
6 30
--> (el brazo se mueve)
<-- DONE
```

5. LIMITACIONES Y MEJORAS

Al revisar el código identificamos los siguientes puntos, que conviene tener presentes al operar el brazo:

- **Control en lazo abierto.** No hay sensores de posición; el código asume que cada servo llega a donde se le ordena.
- **Arranque a 90°.** En `setup()` todos los servos se envían a 90° sin rampa, por lo que el brazo puede dar un movimiento brusco al encender.
- **Error residual.** Por el umbral de 0,5°, el servo puede quedar hasta medio grado de su objetivo.
- **Conteo en modo automático.** El movimiento se activa al recibir seis comandos válidos, sin verificar que correspondan a servos distintos.
- **Acumulación de líneas.** Mientras el brazo se mueve, la pausa entre ciclos puede hacer que lleguen varias líneas al buffer antes de procesarse, y en ese caso se fusionan en una sola. Conviene que el software del computador espere el `DONE` antes de enviar la siguiente secuencia.
- **Rango de pulso común.** El rango de 500 a 2500 µs es el mismo para todos los servos; algunos modelos trabajan mejor entre 1000 y 2000 µs y podrían forzar los topes mecánicos.
- **Sin mensajes de error.** Los comandos inválidos se descartan sin respuesta.

Como mejoras posibles: calibrar el pulso mínimo y máximo de cada servo, agregar una rampa de arranque, responder con un mensaje de error ante comandos inválidos y subir la velocidad serial a 115 200 baudios.

CONCLUSIONES

El sistema cumple con lo necesario para controlar el brazo desde un computador: el PCA9685 libera al Arduino de generar los pulsos, la conversión de ángulos a ticks es

clara y verificable, y los dos modos de operación cubren tanto el control directo como las secuencias coordinadas.

El control de velocidad combina un movimiento lineal para velocidades bajas y uno suavizado para velocidades altas, lo que protege la mecánica del brazo. Sus principales puntos débiles son el arranque sin rampa, la falta de calibración por servo y la ausencia de retroalimentación, aspectos que pueden mejorarse sin cambiar la arquitectura general.

REFERENCIAS

Adafruit Industries. (s. f.). Adafruit PWM Servo Driver Library [Software]. GitHub. https://github.com/adafruit/Adafruit-PWM-Servo-Driver-Library

Arduino. (s. f.). Wire library. Arduino Documentation. https://docs.arduino.cc/language-reference/en/functions/communication/wire/

NXP Semiconductors. (2015). PCA9685: 16-channel, 12-bit PWM Fm+ I2C-bus LED controller (Rev. 4) [Hoja de datos]. https://www.nxp.com/docs/en/data-sheet/PCA9685.pdf